Final Steps & Demo Improvements

Multi-Agent Deep Researcher · Hackathon Completion Guide

Where you are right now

Component	Status	Note
Retriever agent	Working	5 web sources retrieved
PDF chunker	Working	Merges PDF + web sources correctly
Analyst agent	Working	5 claims per source extracted
Critic agent	Working	LLM judge at 0.95 confidence confirmed
Report builder	Working	Markdown + contradiction section
FastAPI server	Working	SSE streaming on port 8000
Streamlit UI	Working	Live agent progress + contradiction view
OpenRouter API key	Fix needed	403 error — top up credits or create new key
Eval harness	Blocked	Will pass once API key is fixed

Remaining steps — in order

1	**Fix OpenRouter API key**	CRITICAL —
2	**Verify API with quick test**	Run the sing
3	**Restart uvicorn with longer timeout**	Prevents str
4	**Re-run evals**	Run_evals.p
5	**Add demo query dropdown to UI**	Removes typ
6	**Add metrics row to UI**	Shows sourc
7	**Pre-cache 2 demo queries**	Run cache_

8	**Rehearse demo twice**	Time it — m
9	**Prepare 5 slides**	Architecture,

Step 1 in detail — fix OpenRouter 403

Go to openrouter.ai/credits — top up $5 minimum, or create a new key at openrouter.ai/keys. Then update your .env file:

```
OPENROUTER_API_KEY=sk-or-v1-YOUR-NEW-KEY-HERE
TAVILY_API_KEY=tvly-YOUR-KEY-HERE
```

Then restart uvicorn with extended timeout:

```
uvicorn main:app --reload --port 8000 --timeout-keep-alive 180
```

Then verify the fix worked:

```
python -c "
from dotenv import load_dotenv; load_dotenv()
import requests, json
r = requests.post('http://localhost:8000/research', data={'query': 'GPT-4 test'}, stream=True,
timeout=60)
for line in r.iter_lines():
if line and b'status' in line:
p = json.loads(line[6:])
print('[' + p['status'] + ']')
"
```

Expected output:

```
[retriever]
[pdf_chunker]
[analyst]
[critic]
[report_builder]
```

Demo improvements — highest impact first

Improvement	Impact	What to do
Add query dropdown	HIGH	Replace free-text input with a selectbox of pre-tested queries. Eliminates typing errors live. Add to app.py sidebar above the text area.

Add metrics row	HIGH	Show st.metric() for sources analysed, contradictions found, and top confidence score. Judges see the numbers instantly without reading the report.
Pre-cache demo results	HIGH	Run cache_demo.py to save 2 query results to JSON. If the live query hangs, load from cache instantly with a single flag.
Cap sources to 6	HIGH	In critic.py add sources = state['sources'][:6]. Cuts critic LLM calls by 55% and prevents timeout during the demo.
Cap claims to 3	MEDIUM	In analyst.py change prompt to 'Extract up to 3 claims'. Further reduces critic pairs from 225 to 81 comparisons.
Show contradiction count in agent status	MEDIUM	Update the agent progress panel to show 'critic — 3 contradictions found' as it completes. More informative than just a tick.
Add PDF filename to sources	MEDIUM	In pdf_chunker.py use the actual uploaded filename instead of 'uploaded.pdf'. Makes citations look professional in the report.
Export report as PDF	MEDIUM	Add a download button in app.py that converts report_markdown to PDF using reportlab. Judges can take away a copy.
Add a confidence histogram	LOW	Use st.bar_chart() to show distribution of contradiction confidence scores. One extra line of code, looks impressive.
Domain label in subtitle	LOW	Change the subtitle in app.py from generic to 'AI/ML research' to show intentional scoping — judges notice this.

Code snippets — copy paste ready

1. Demo query dropdown — add to app.py sidebar

```
demo_queries = {
"Select a demo question...": "",
"GPT-4 MMLU accuracy": "What accuracy does GPT-4 achieve on the MMLU benchmark?",
"RAG vs Fine-tuning": "How does RAG compare to fine-tuning for domain adaptation?",
"LLM energy consumption": "What are energy consumption differences between GPT-3 and GPT-4?",
}
selected = st.selectbox("Or pick a demo question", list(demo_queries.keys()))
query = st.text_area("Research question", value=demo_queries[selected], height=100)
```

2. Metrics row — add to app.py after report renders

```
if final_contradictions:
m1, m2, m3 = st.columns(3)
m1.metric("Sources analysed", len(final_state.get("sources", [])))
m2.metric("Contradictions found", len(final_contradictions))
m3.metric("Top confidence", f"{max(c['confidence'] for c in final_contradictions):.0%}")
```

3. Cap sources in critic.py — add as first line of run_critic()

```
def run_critic(state: ResearchState) -> dict:
contradictions: list[Contradiction] = []
sources = state["sources"][:6] # cap to prevent timeout
```

4. PDF download button — add to app.py after report renders

```
st.download_button(
label="Download report as markdown",
data=final["report_markdown"],
file_name="research_report.md",
mime="text/markdown",
)
```

Demo script — 4 minutes

0:00	**Open browser to localhost:8501**	Have it ready before you start talking. Show the clean UI.
0:20	**Explain the problem in one sentence**	"Research agents today retrieve sources but don't tell you when those sources disagree with each other. Ours does."
0:40	**Select demo query from dropdown**	Pick GPT-4 MMLU accuracy. Click Run research. Talk through the agents ticking live.
1:30	**Point to contradiction panel**	"This is our differentiator — the Critic agent compared every claim across all sources and found X conflicts. Here's the highest confidence one at 95%."
2:00	**Show the side-by-side conflict view**	"Source A says 90%, Source B says 86.4%. Both can't be right. The agent explains why — different evaluation protocols."
2:30	**Show the metrics row**	"9 sources analysed, 3 contradictions found, top confidence 95%. That's the system working."
3:00	**Show eval score**	"We ran 3 test cases with known good answers. Scored X/9. Here's the architecture that makes it work."
3:30	**Show architecture slide**	LangGraph state machine, 5 agents, LLM-as-judge. Keep it to 30 seconds.
3:50	**Close with next steps**	"In production we'd add memory across sessions, a domain selector, and export to Notion or Confluence."

5 slides you need

Slide 1	**The problem**	Researchers waste hours cross-checking sources manually. AI tools retrieve but don't validate. One line: 'We built an agent that reads your sources and tells you when they disagree.'
Slide 2	**Architecture**	LangGraph state machine diagram. 5 agents. Tavily + PDF as equal sources. LLM-as-judge critic. One flow arrow showing query in, report out.

Slide 3	**Live demo screenshot**	Screenshot of your contradiction panel from the UI. Circle the confidence score and the side-by-side conflict view in red.
Slide 4	**Eval results**	Show your eval score (X/9). Bullet: what the 3 test cases were. Bullet: what correct looks like. This is your proof it works beyond the demo.
Slide 5	**What's next**	Memory across sessions. Domain selector (medical, legal, financial). Export to Notion/Confluence. Multi-language support. 30-second slide, not a roadmap.

> The contradiction panel side-by-side is your entire demo. Every other feature supports it. When you present, pause on that screen for 20 seconds and let judges read it. Don't rush past the thing that makes you different.